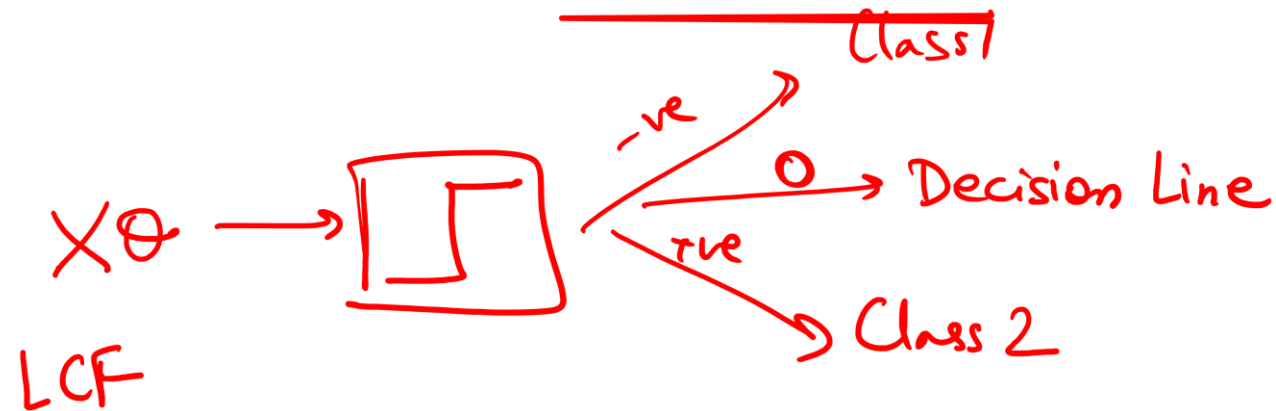


# Support Vector Machine

Nimisha Roy  
Georgia Tech

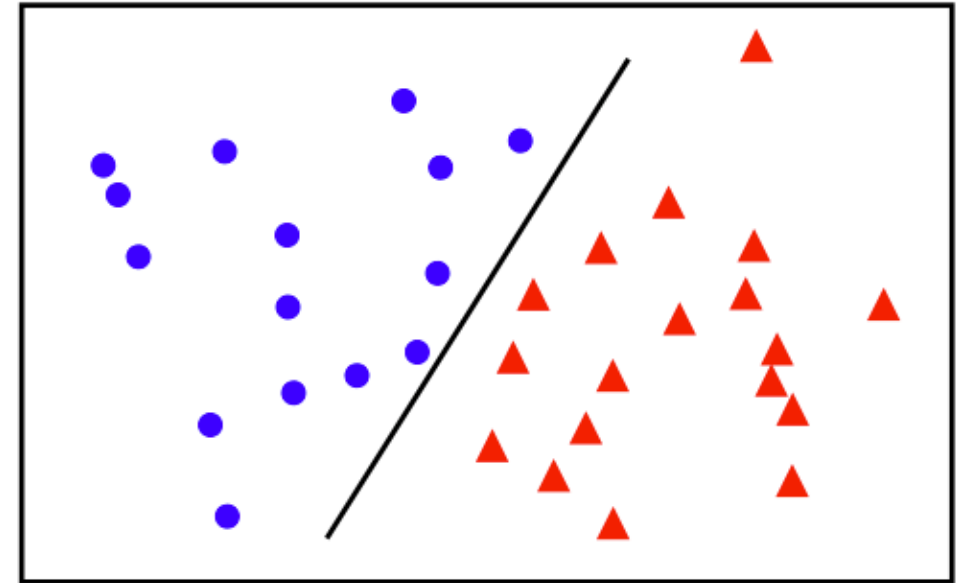
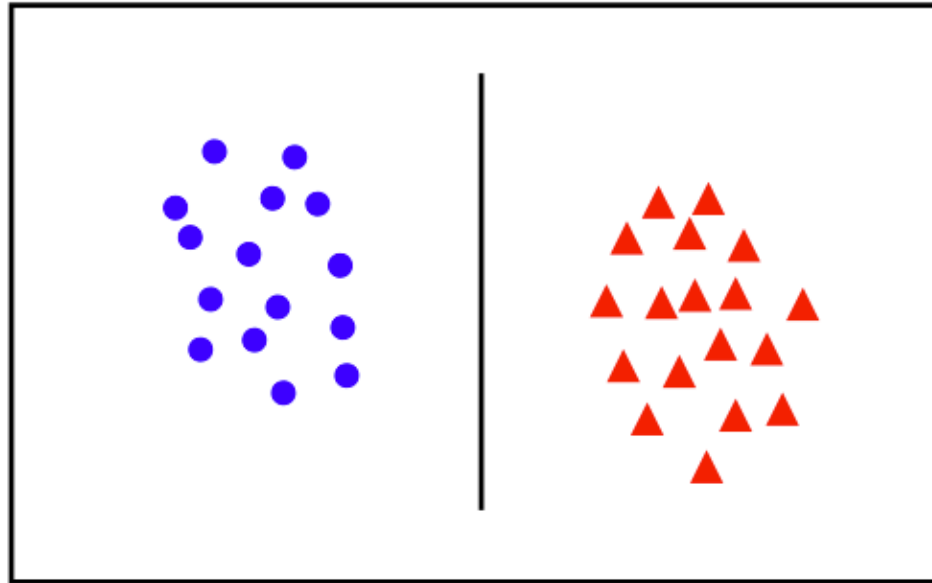
# Outline

- Precursor: Linear Classifier and Perceptron ←
- Support Vector Machine
- Parameter Learning

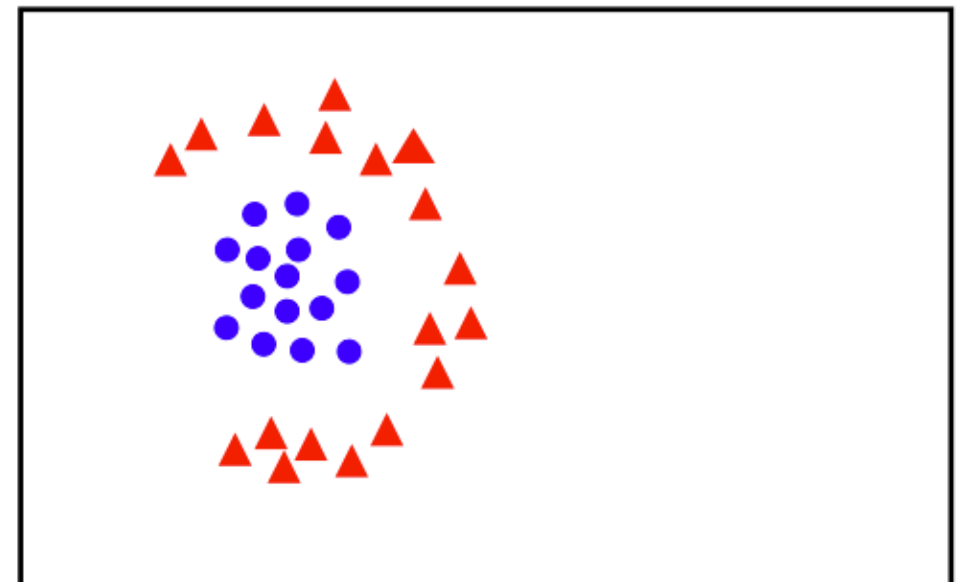
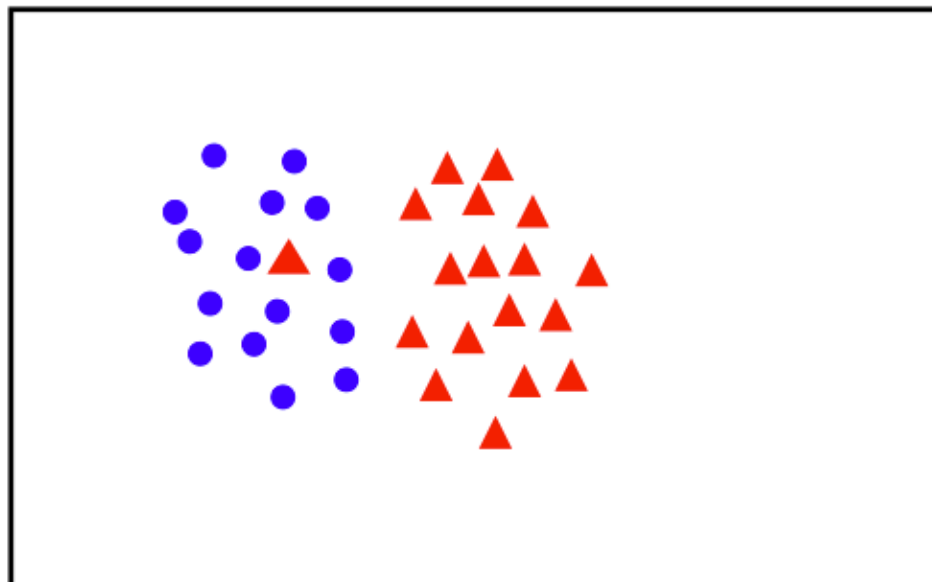


# Linear Separability

linearly  
separable



not  
linearly  
separable



# Binary Classification

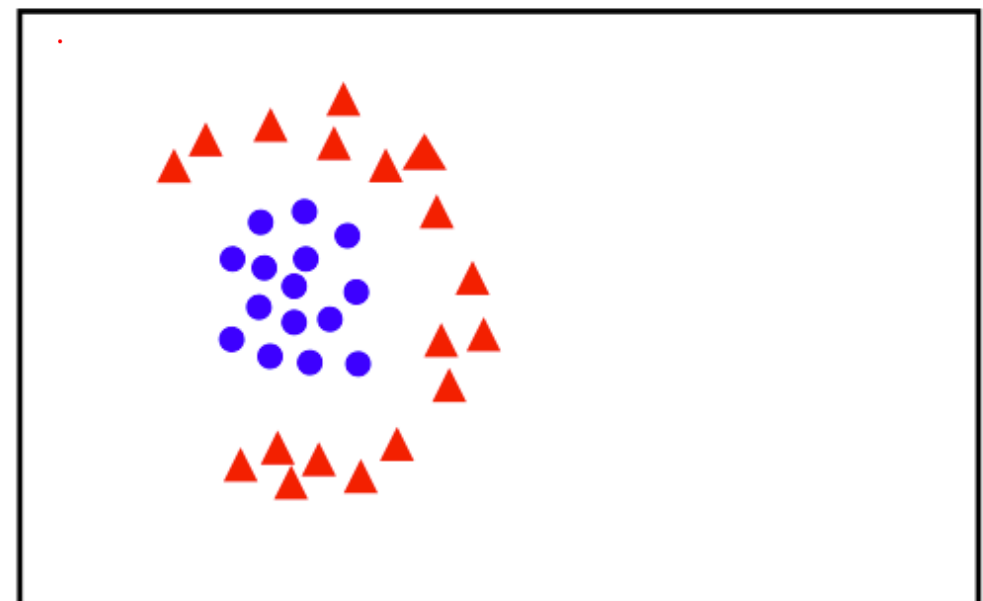
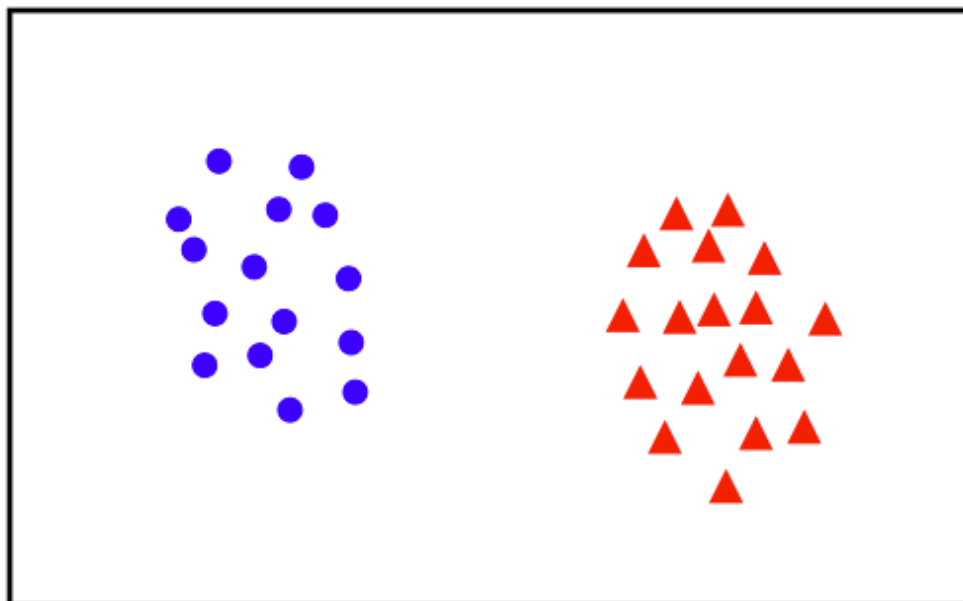
Given training data  $(x^{i}, y^{i})$  for  $i = 1 \dots N$ , with  $x^{i} \in \mathbb{R}^d$  and  $y_i \in \{-1, 1\}$ , learn a classifier  $f(\mathbf{x})$  such that

$$f(x^{i}) \begin{cases} \geq 0 \\ < 0 \end{cases}$$

+1

-1

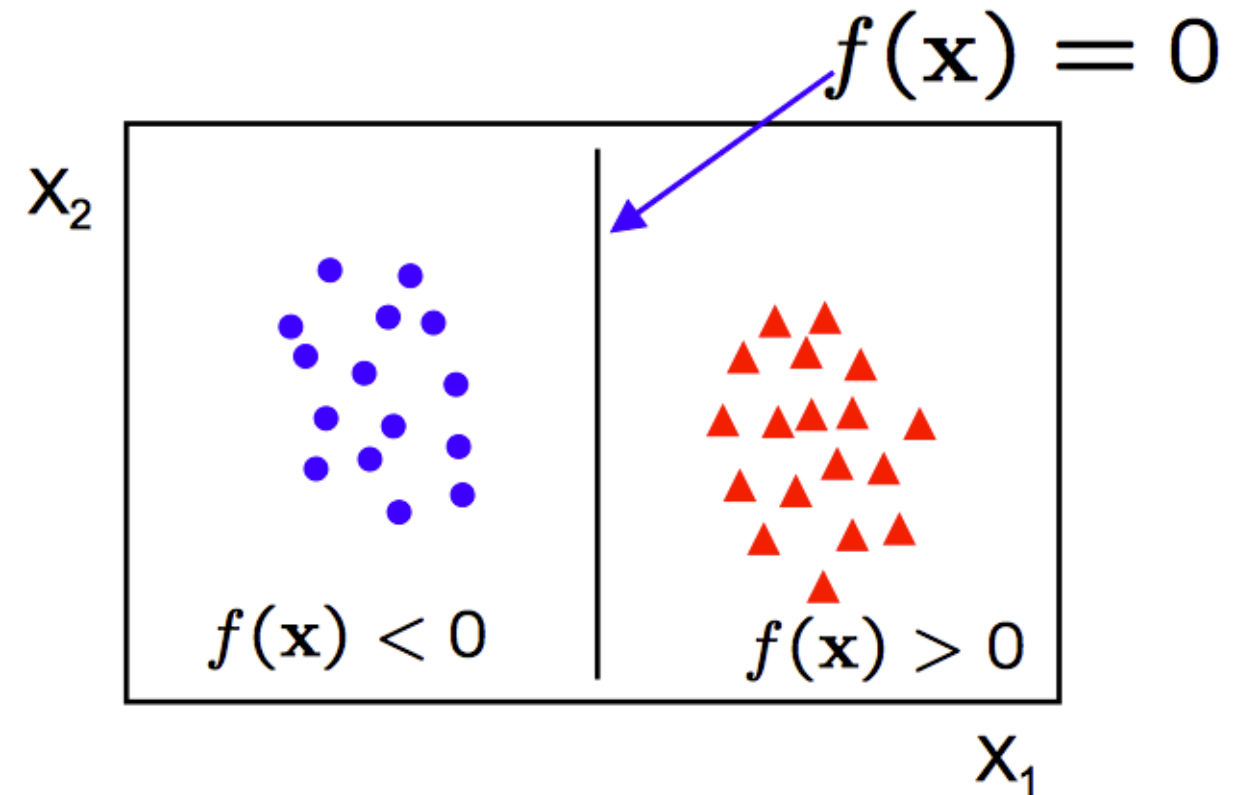
i.e.  $y^{i} f(x^{i}) > 0$  for a correct classification.



# Linear Classifier

A linear classifier has the form

$$f(x) = x\theta + \theta_0$$

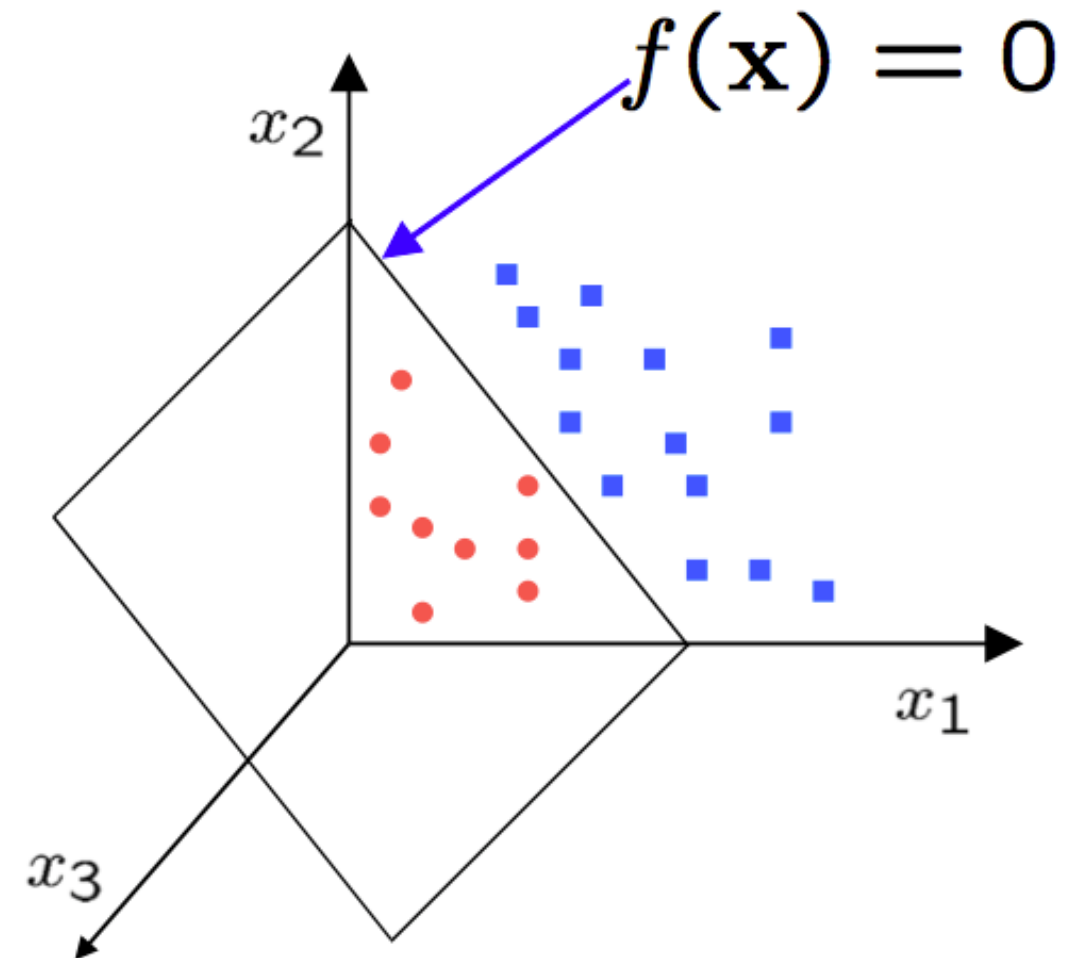


- in 2D the discriminant is a line
- $\theta$  is the **normal** to the line, and  $\theta_0$  the **bias**
- $\theta$  is known as the **weight vector**

# Linear Classifier (higher dimension)

A linear classifier has the form

$$f(x) = x\theta + \theta_0$$



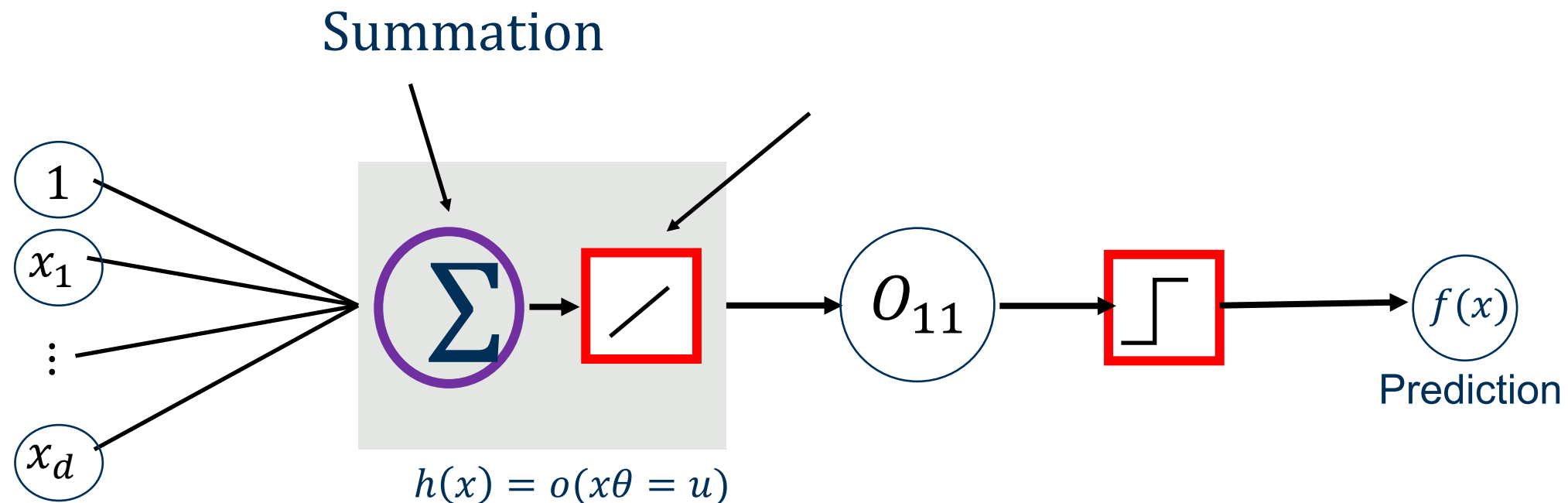
- in 3D the discriminant is a plane, and in nD it is a hyperplane

# The Perceptron Classifier

Considering  $\mathbf{x}$  is linearly separable and  $\mathbf{y}$  has two labels of  $\{-1, 1\}$

$$f(x^{\{i\}}) = x^{\{i\}} \theta \quad \text{Bias is inside } \theta \text{ now}$$

How can we separate datapoints with label 1 from datapoints with label -1 using a line?





# The Perceptron Classifier

## Perceptron Algorithm:

- Initialize  $\theta$
- Go through each datapoint  $\{x^{(i)}, y^{(i)}\}$
- If  $x^{(i)}$  is misclassified then  $\theta^{t+1} \leftarrow \theta^t + \alpha y^{(i)} x^{(i)}$
- Until all datapoints are correctly classified

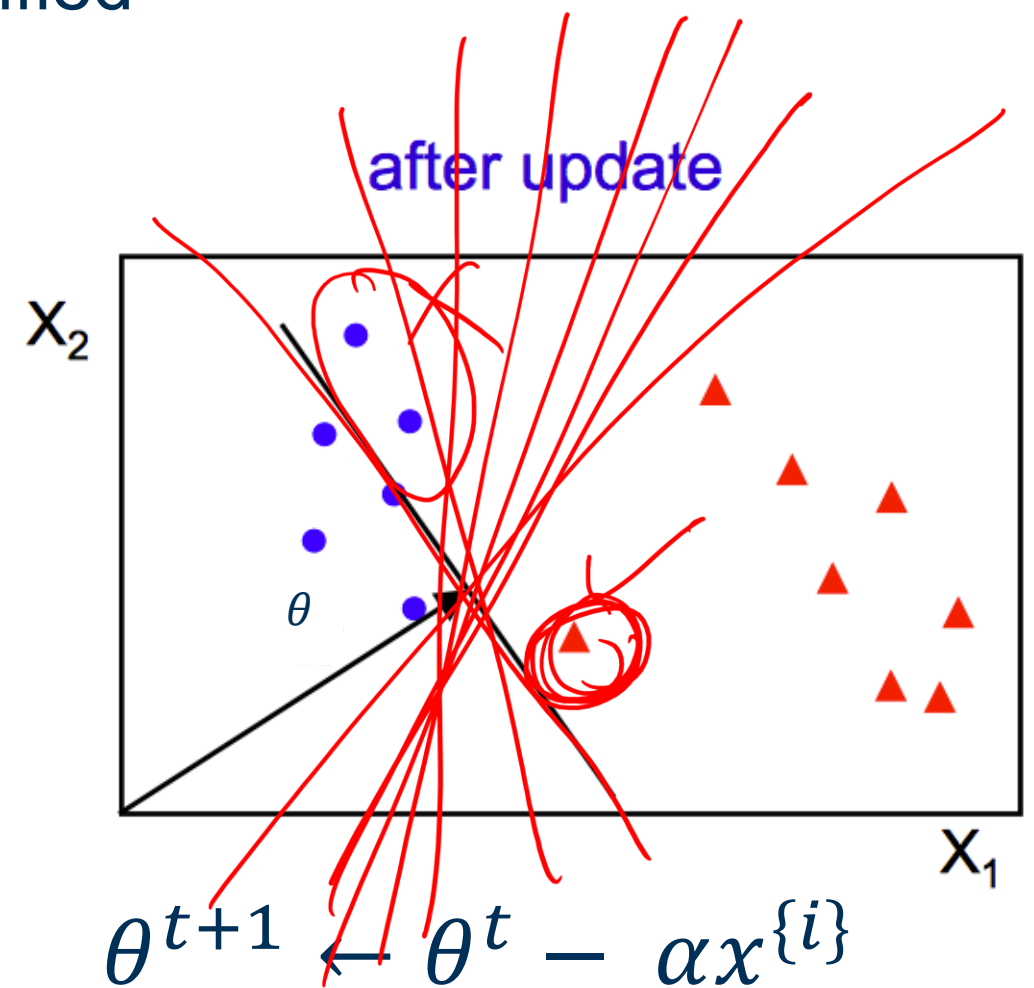
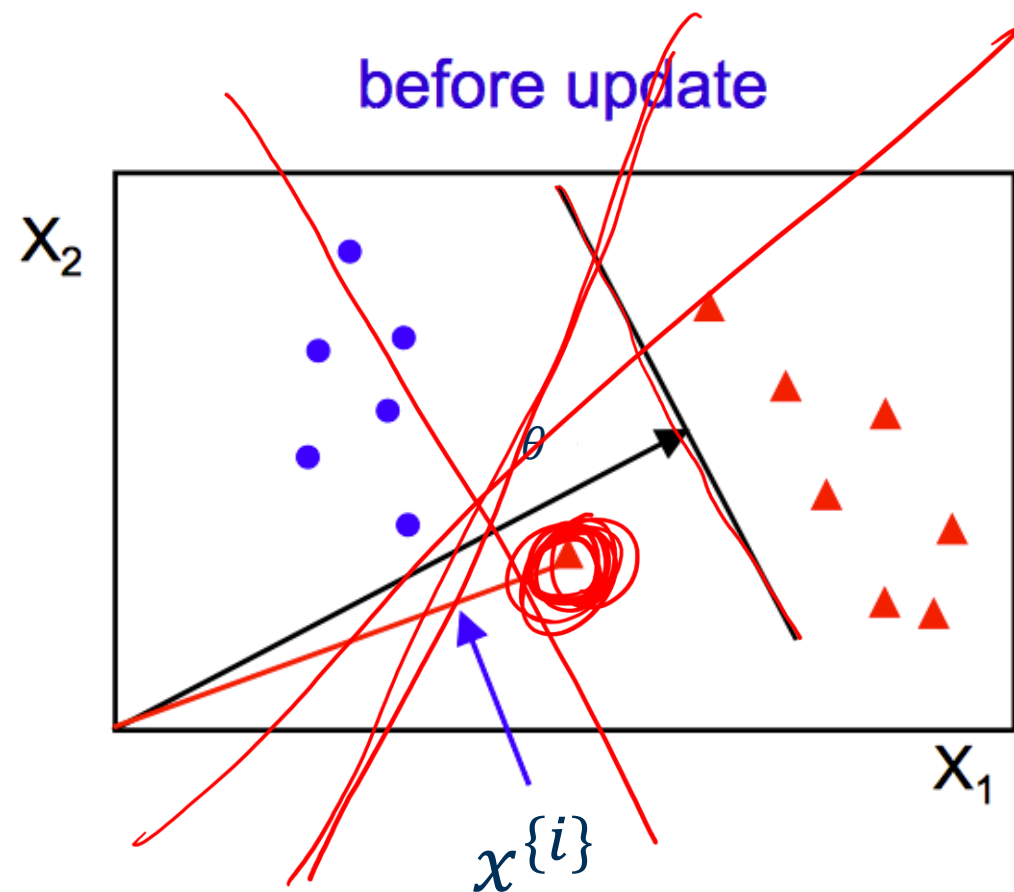
Misclassified

Ex.  $y^{(i)} f(x^{(i)}) < 0$

actual      predicted



- Initialize  $\theta$
- Go through each datapoint  $\{x^{(i)}, y^{(i)}\}$
- If  $x^{(i)}$  is misclassified then  $\theta^{t+1} \leftarrow \theta^t + \alpha y^{(i)} x^{(i)}$
- Until all datapoints are correctly classified



# Recap

- Perceptron Algo

GD  
SGD

It works only  
for linearly  
separable  
data

Step 0: Initialize  $\theta \neq 0$

Step 1:  $x^{(i)} \cdot \theta \rightarrow$    $\rightarrow f(x^{(i)})$

$f(x^{(i)})$  is a function that outputs a value. If the value is positive (+ve), it is classified as Class 1. If the value is negative (-ve), it is classified as Class 2. The value 0 is the Decision Line.

Step 2: Misclassified or not.

$y^{(i)} \cdot f(x^{(i)}) < 0 \rightarrow$  misclassified  
 $> 0 \rightarrow$  classified correctly

Step 3: Nudge  $\theta$ .

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \alpha \cdot \nabla_{\theta} \text{Objective}$$
$$\theta^{(t+1)} \leftarrow \theta^{(t)} + \alpha \cdot y^{(i)} \cdot x^{(i)}$$

Step 4: Keep on repeating until all datapoints are correctly classified

Objective function:  $-\sum y^{(i)} \theta^T x^{(i)}$

# Perceptron update

If  $x^{(i)}$  is misclassified then  $\theta^{t+1} \leftarrow \theta^t + \alpha y^{(i)} x^{(i)}$

Misclassified point pulls  $\theta$  in the direction that will flip the sign of  $\theta \cdot x_i$  next time.

## 1. Multiply by $y_i$ :choose the correct direction

- $y_i = +1$  :move  $\theta$  **toward**  $x_i$  — increase  $\theta \cdot x_i$
- $y_i = -1$  :move  $\theta$  **away** from  $x_i$  — decrease  $\theta \cdot x_i$

This flips the sign automatically so the dot product moves in the correct direction.

## 2. Multiply by $x_i$ :fix the boundary using the misclassified point

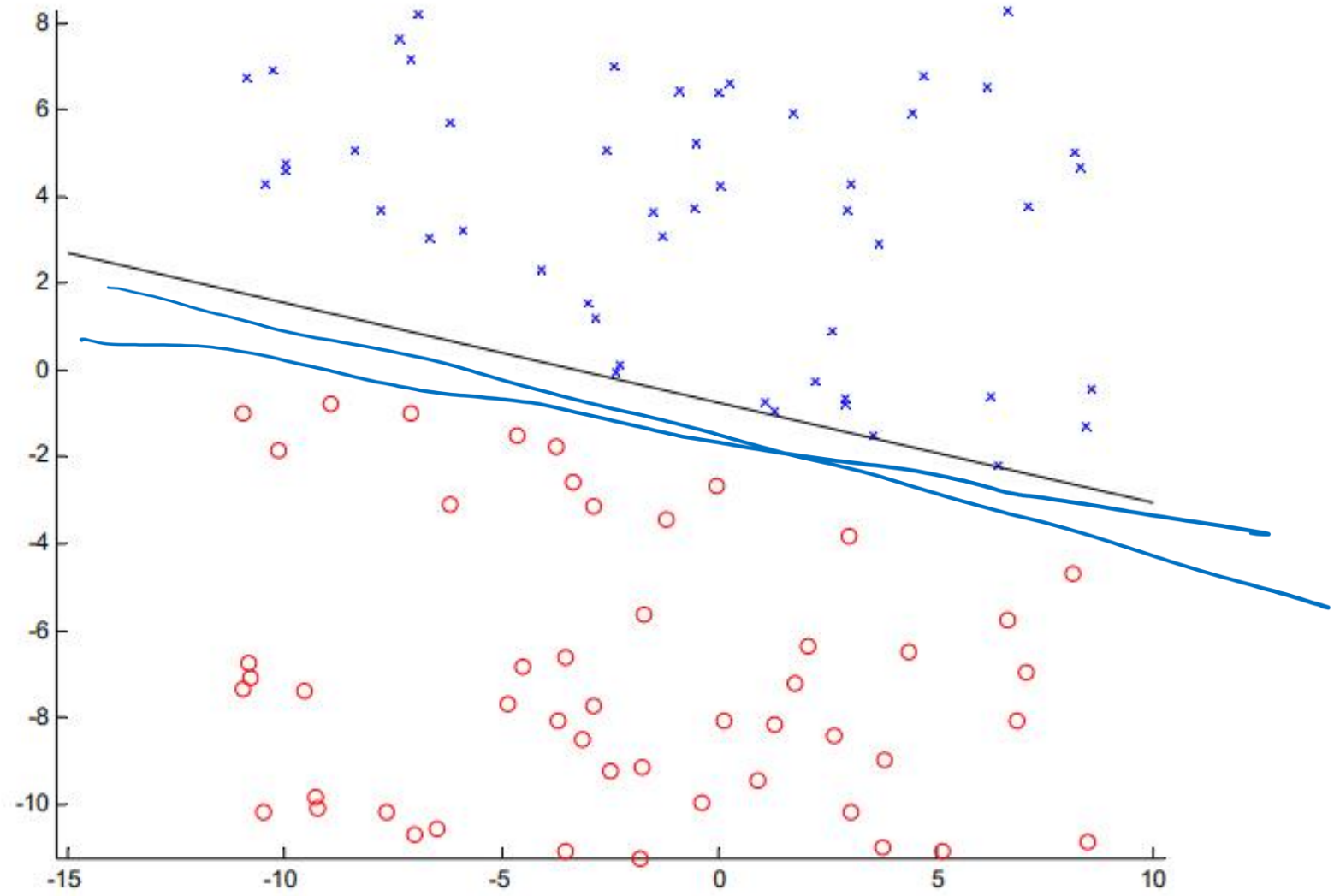
- $x_i$  points directly to *where* the mistake occurred
- Adding  $+\alpha x_i$  rotates the decision boundary toward that point
- Adding  $-\alpha x_i$  rotates it away from that point

The model adjusts based on the geometry of that specific data point.

## 3. Multiply by $\alpha$ : control the step size

- Small  $\alpha \rightarrow$  gentle correction. Large  $\alpha \rightarrow$  aggressive boundary movement

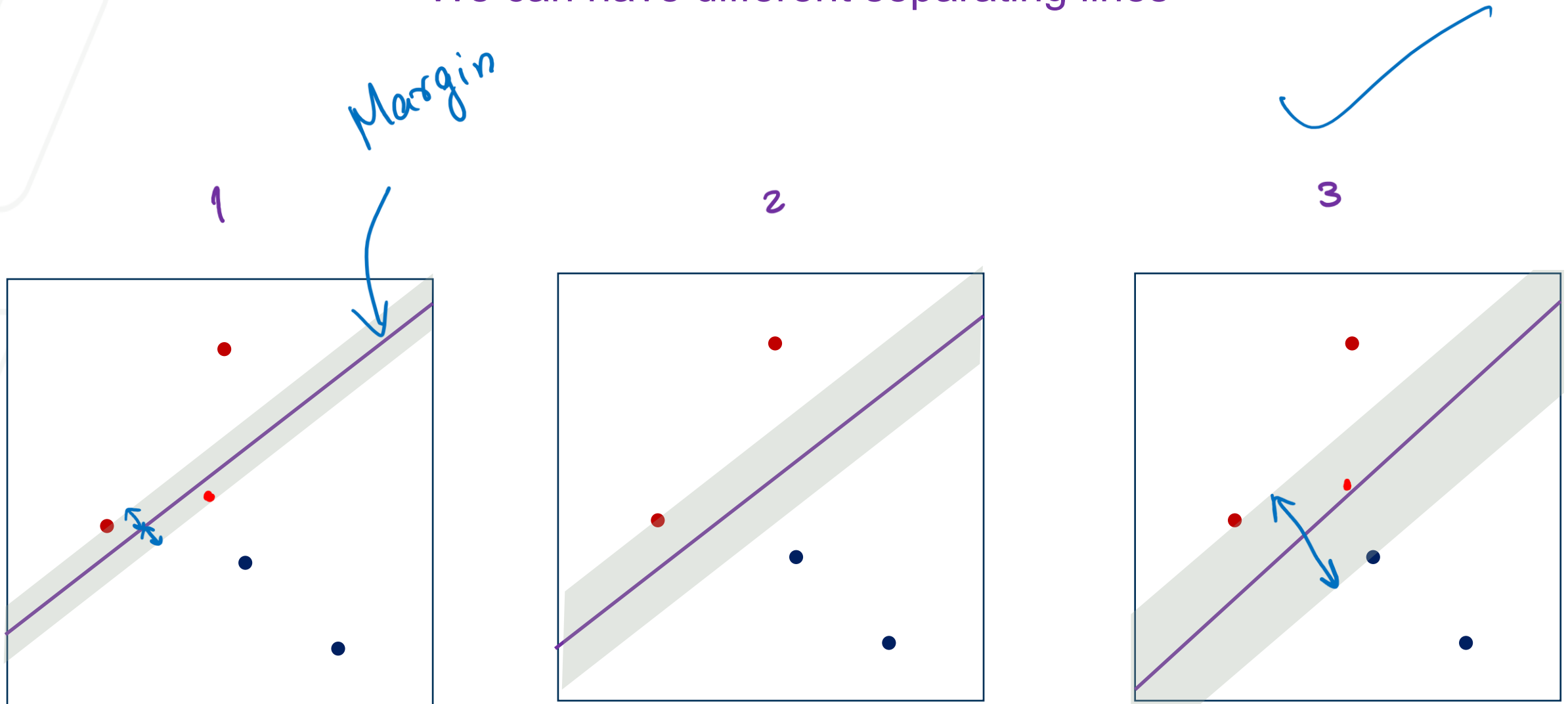
## Perceptron example



- if the data is linearly separable, then the algorithm will converge
- convergence can be slow ...
- separating line close to training data
- we would prefer a larger margin for generalization (better generalization)

# Linear separation

We can have different separating lines



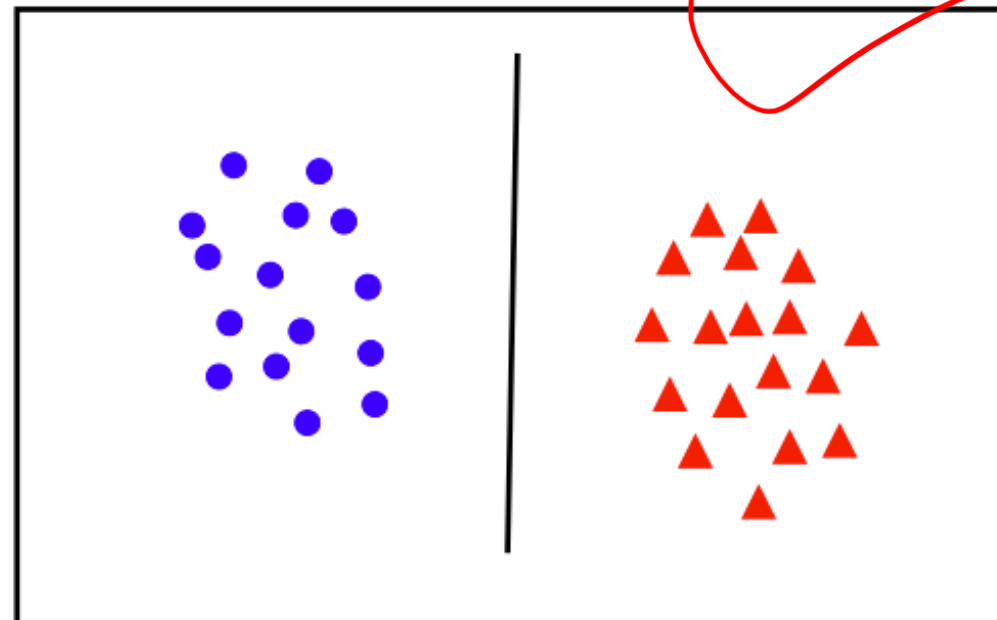
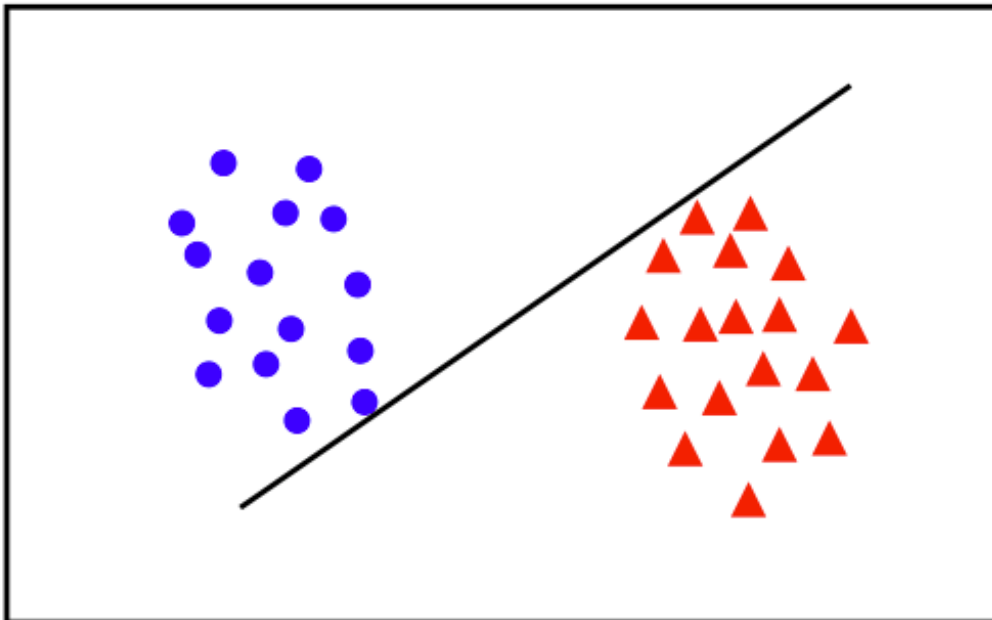
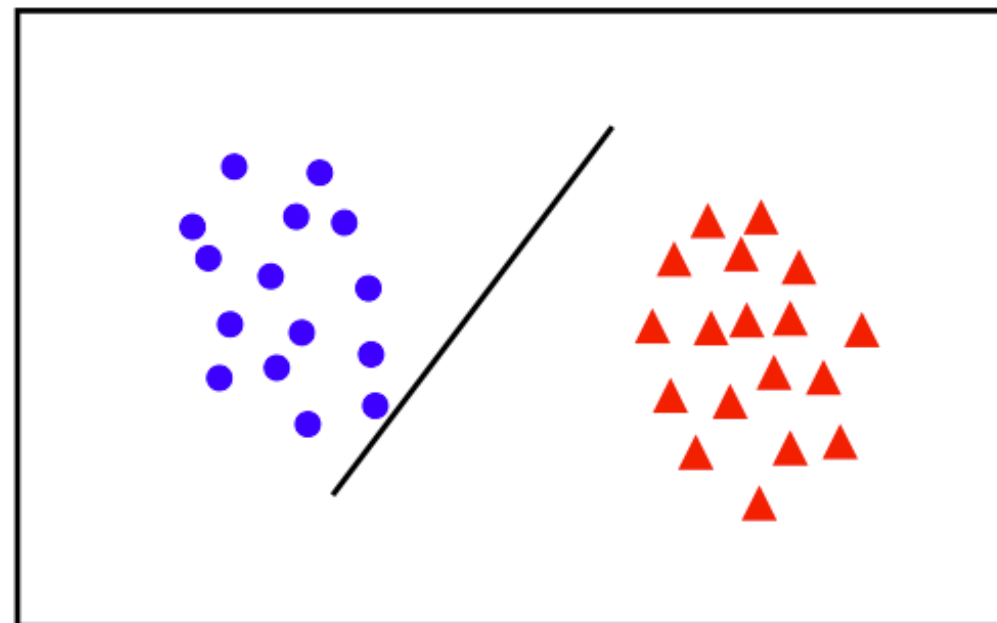
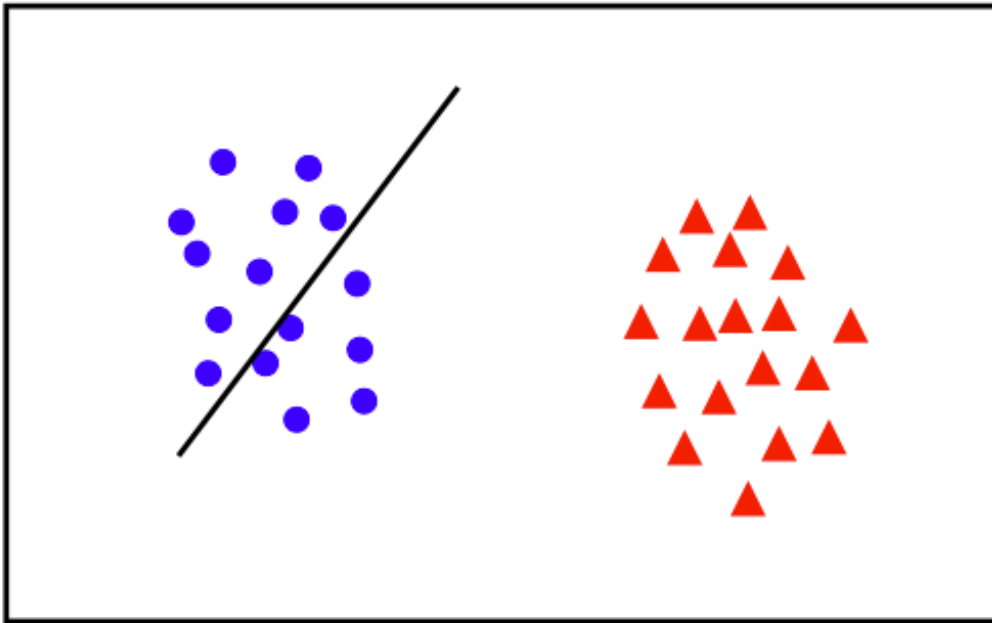
Which line is the best?

Why is the bigger margin better?

What  $\theta$  maximizes the margin?

All cases, error is zero and they are linear, so they are all good for generalization.

# What is the Best $\theta$ ?



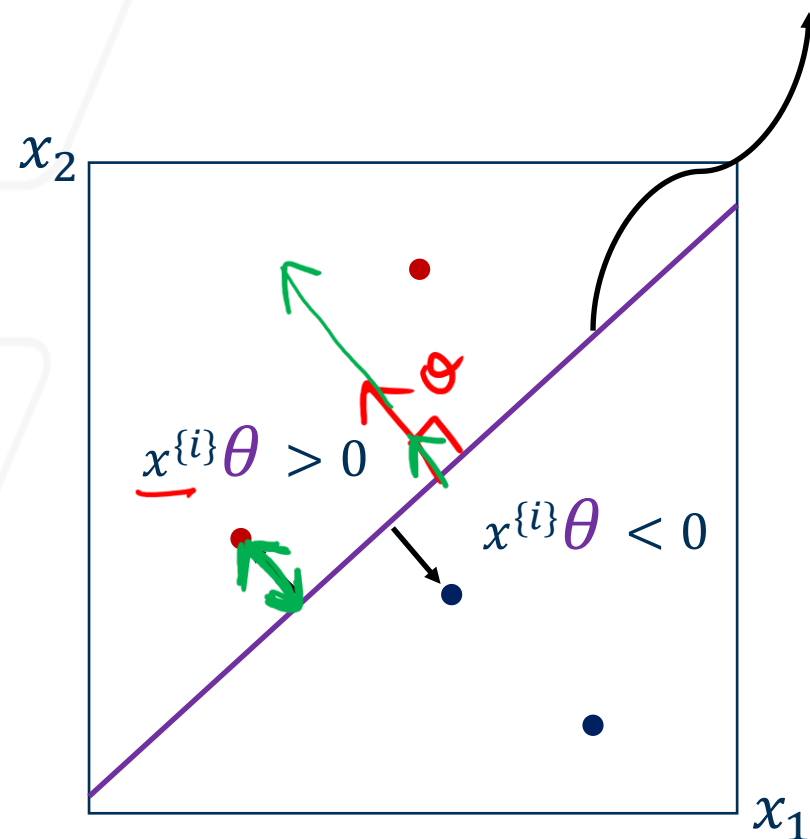
# Outline

- Precursor: Linear Classifier and Perceptron
- Support Vector Machine ←
- Parameter Learning ←



# Finding $\theta$ with a fat margin

Solution (decision boundary) of the line:  $x\theta = 0$



Let  $x^{(i)}$  to be the nearest data point to the line (plane):

$$|x^{(i)}\theta| > 0$$

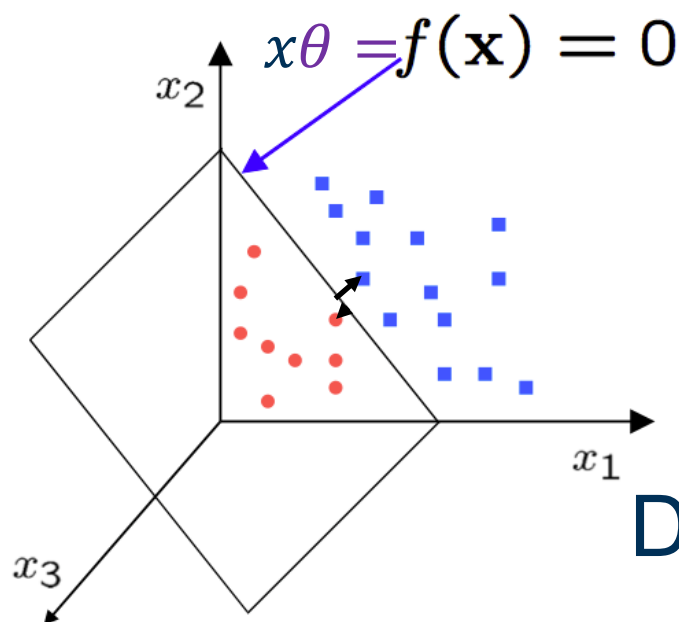
Our line solution is  $x\theta = 0$

Does it matter if I scale up or down  $\theta$  for the decision boundary?

$$|x^{(i)}\theta| = 1 \rightarrow \text{normalization}$$

Let's pull out  $\theta_0$  from  $\theta = (\theta_1, \dots, \theta_d)$  and call it be  $b$

Decision boundary would be:  $x\theta + b = 0$





# Computing the distance

The distance between  $x^{\{i\}}$  and the line  $x\theta + b = 0$  where  $|x^{\{i\}}\theta + b| = 1$

The vector  $\theta$  is perpendicular to the decision line.

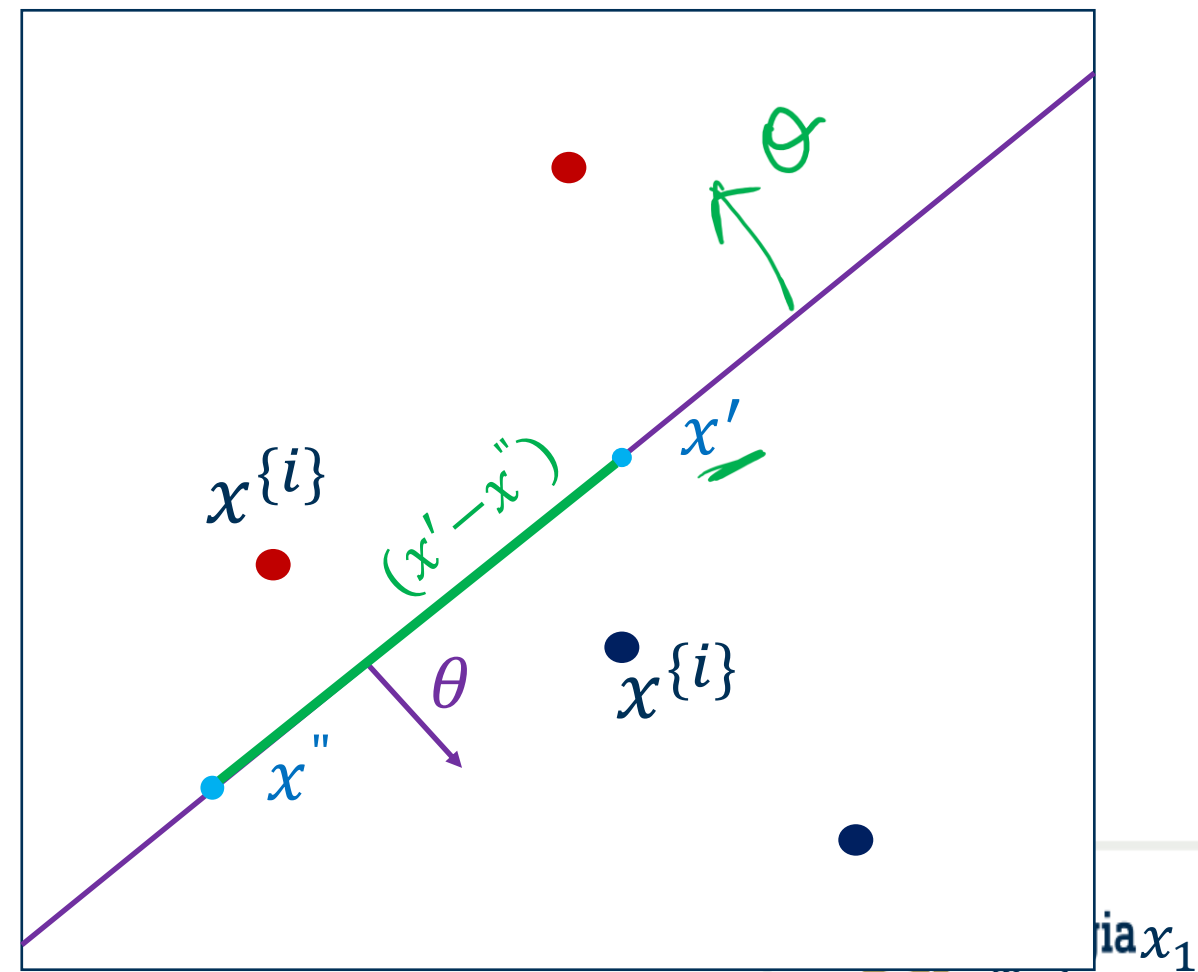
Consider  $x'$  and  $x''$  on the plane

$$\underbrace{x'\theta + b = 0}_{\text{green underline}} \quad \text{and} \quad \underbrace{x''\theta + b = 0}_{\text{green underline}}$$

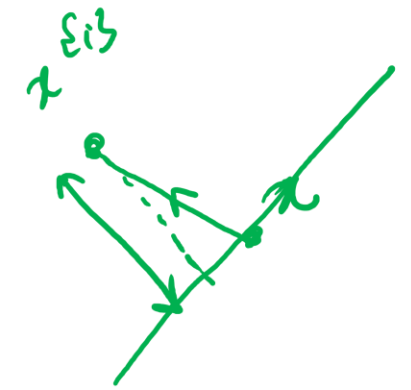
$$\downarrow$$
$$x'\theta + b = x''\theta + b$$

$$\Rightarrow \underbrace{(x' - x'')\theta = 0}_{\text{green underline}}$$

$x_2$



ia  $x_1$

$$(x - x^{(i)}) \cdot \hat{\theta}$$


# Now we need to maximize the margin: Objective Function

Maximize  $\frac{2}{||\theta||}$

Subject to Min value of  $|x^{(i)}\theta + b| = 1 \Rightarrow \text{nearest neighbour}$   
 $i = 1, 2, \dots, N$

*N constraints  
non-differentiable*

There is a “min” in our constraining; it can be hard to optimize this problem(non-convex form)

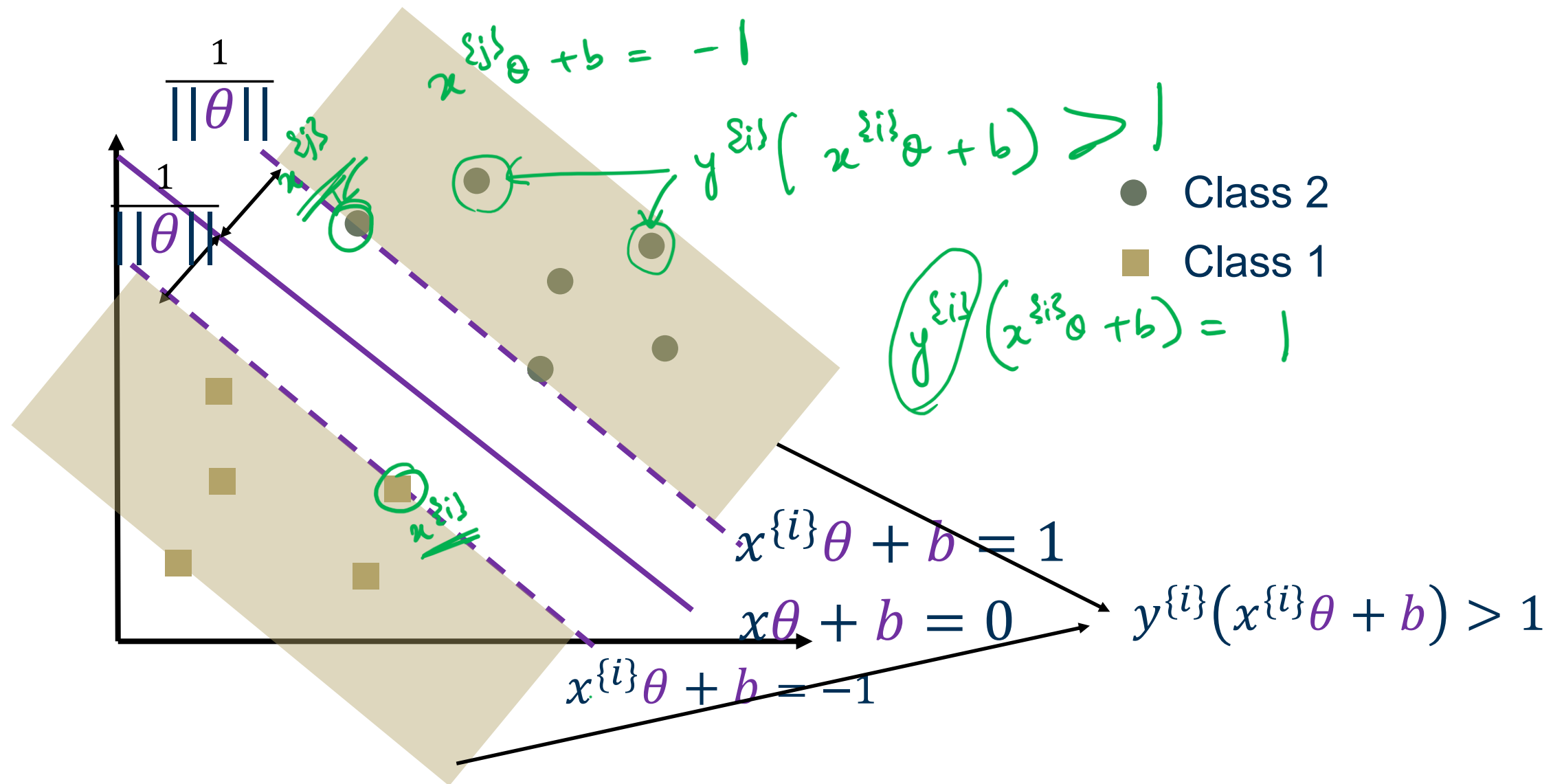
Can I write the following term to get rid of absolute value?

$$|x^{(i)}\theta + b| = y^{(i)}(x^{(i)}\theta + b) \Rightarrow \text{for a correct classification}$$

Maximize  $\frac{2}{||\theta||}$

Subject to  $y^{(i)}(x^{(i)}\theta + b) \geq \underline{1}$  for  $i = 1, 2, \dots, N$

*equality holds when closest point*



Maximize  $\frac{2}{\|\theta\|}$

If  $\theta \neq 0$ , there exists a max value

Subject to  $y^{i}(x^{i}\theta + b) \geq 1$  for  $i = 1, 2, \dots, N$

Minimize  $\frac{1}{2}\theta\theta^T$

Subject to  $y^{i}(x^{i}\theta + b) \geq 1$  for  $i = 1, 2, \dots, N$

Inequality Constraint

# Constrained optimization

$$\text{Minimize } \frac{1}{2} \theta \theta^T$$

$$\text{Subject to } \underline{y^{\{i\}}(x^{\{i\}}\theta + b) \geq 1 \text{ for } i = 1, 2, \dots, N}$$

$$\theta \in \mathbb{R}^d, b \in \mathbb{R}$$

Using Lagrange method:

But wait, there is an **inequality** in our constraints

We use Karush-Kuhn-Tucker (KKT) condition to deal with this problem

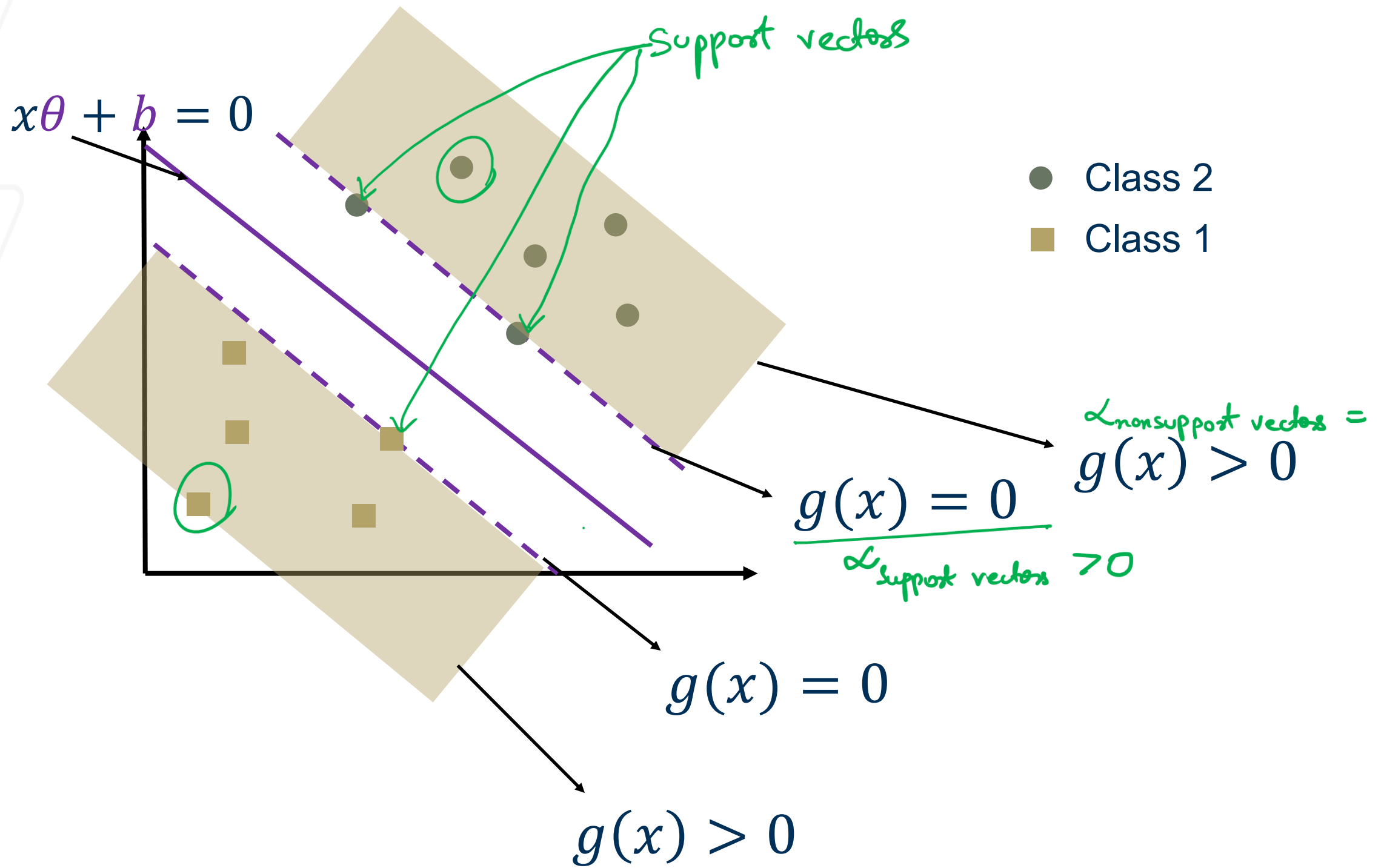
$$\underline{g(x)} = y^{\{i\}}(x^{\{i\}}\theta + b) - 1 \quad \begin{matrix} g(x) \geq 0 \\ \alpha = \text{lagrange multiplier} \end{matrix}$$

We need to optimize

$\theta, b$ , and  $\alpha$

- 1) Stationary
- 2)  $g(x) \geq 0$  Primal feasibility
- 3)  $\alpha \geq 0$  Dual feasibility
- 4)  $g(x)\alpha = 0$  Complementary slackness

$$\Rightarrow \begin{cases} g(x) > 0, & \alpha = 0 \\ \alpha > 0, & g(x) = 0 \end{cases}$$



$$g(x) = y^{(i)}(x^{(i)}\theta + b) - 1$$

3)  $g(x)\alpha = 0$  Complementary slackness

$$\Rightarrow \begin{cases} g(x) > 0, & \alpha = 0 \\ \alpha > 0, & g(x) = 0 \end{cases}$$

# Lagrange formulation (Primal $\rightarrow$ Dual Form)

$$\text{Minimize } \frac{1}{2} \theta \theta^T \quad \text{s.t.} \quad \underline{y^{\{i\}}(x^{\{i\}} \theta + b) - 1 \geq 0}$$

$$\mathcal{L}(\theta, b, \alpha) = \frac{1}{2} \theta \theta^T - \sum_{i=1}^N \alpha_i (y^{\{i\}}(x^{\{i\}} \theta + b) - 1)$$

*primal* (handwritten red arrow pointing to  $\theta$ )

Minimize w.r.t  $\theta$  and  $b$  and maximize w.r.t  $\alpha_i$  *we have N constraints*

$$\nabla_{\theta} \mathcal{L}(\theta, b, \alpha) = \theta - \sum_{i=1}^N \alpha_i y^{\{i\}} x^{\{i\}} = 0$$

$$\nabla_b \mathcal{L}(\theta, b, \alpha) = - \sum_{i=1}^N \alpha_i y^{\{i\}} = 0$$



$$\theta = \sum_{i=1}^N \alpha_i y^{(i)} x^{(i)}$$

$$\sum_{i=1}^N \alpha_i y^{(i)} = 0$$

Let's substitute these in the Lagrangian:

$$\mathcal{L}(\theta, b, \alpha) = \frac{1}{2} \theta \theta^T - \sum_{i=1}^N \alpha_i (y^{(i)} (x^{(i)} \theta + b) - 1)$$

$$\mathcal{L}(\theta, b, \alpha) = \sum_{i=1}^N \alpha_i + \frac{1}{2} \theta \theta^T - \sum_{i=1}^N \alpha_i (y^{(i)} (x^{(i)} \theta + b))$$

$$\begin{aligned} \sum_i \alpha_i y_i b \\ = b \sum_i \alpha_i y_i \\ = 0 \end{aligned}$$

$$\mathcal{L}(\theta, b, \alpha) = \sum_{i=1}^N \alpha_i + \frac{1}{2} \theta \theta^T - \sum_{i=1}^N \alpha_i (y^{(i)} (x^{(i)} \theta))$$

$$= \sum_{i=1}^N \alpha_i + \frac{1}{2} \theta \theta^T - \theta \theta^T = \sum_{i=1}^N \alpha_i - \frac{1}{2} \theta \theta^T$$



$$\theta = \sum_{i=1}^N \alpha_i y^{(i)} x^{(i)}$$

$$\sum_{i=1}^N \alpha_i y^{(i)} = 0$$

$$\mathcal{L}(\theta, b, \alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \theta \theta^T$$

$$\mathcal{L}(\theta, b, \alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y^{(i)} y^{(j)} \alpha_i \alpha_j x^{(i)} x^{(j)T}$$

maximize w.r.t each  $\alpha_i \geq 0$  for  $i = 1, \dots, N$

and

$$\sum_{i=1}^N \alpha_i y^{(i)} = 0$$

# The solution – quadratic programming

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y^{(i)} y^{(j)} \alpha_i \alpha_j x^{(i)} x^{(j)T}$$

Quadratic programming packages usually use “min”

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y^{(i)} y^{(j)} \alpha_i \alpha_j x^{(i)} x^{(j)T} - \sum_{i=1}^N \alpha_i$$

$$\min_{\alpha} \frac{1}{2} \alpha^T \begin{bmatrix} y^{(1)} y^{(1)} x^{(1)} x^{(1)T} & \dots & y^{(1)} y^{(N)} x^{(1)} x^{(N)T} \\ \vdots & \ddots & \vdots \\ y^{(N)} y^{(1)} x^{(N)} x^{(1)T} & \dots & y^{(N)} y^{(N)} x^{(N)} x^{(N)T} \end{bmatrix} \alpha + (-I^T) \alpha$$

Handwritten annotations in red:

- $1 \times 1$  (above  $y^{(1)} y^{(1)} x^{(1)} x^{(1)T}$ )
- $1 \times 1$  (above  $y^{(1)} y^{(N)} x^{(1)} x^{(N)T}$ )
- $1 \times d$  (above  $y^{(1)} y^{(N)} x^{(1)} x^{(N)T}$ )
- $d \times 1$  (above  $y^{(N)} y^{(N)} x^{(N)} x^{(N)T}$ )
- $1 \times n$  (below  $\alpha^T$ )
- $n \times n$  (below the matrix)
- $n \times 1$  (below the vector  $\alpha$ )

$$\min_{\alpha} \frac{1}{2} \alpha^T \begin{bmatrix} y^{\{1\}} y^{\{1\}} x^{\{1\}} x^{\{1\}T} & \dots & y^{\{1\}} y^{\{N\}} x^{\{1\}} x^{\{N\}T} \\ \vdots & \ddots & \vdots \\ y^{\{N\}} y^{\{1\}} x^{\{N\}} x^{\{1\}T} & \dots & y^{\{N\}} y^{\{N\}} x^{\{N\}} x^{\{N\}T} \end{bmatrix} \underbrace{\alpha + (-I^T) \alpha}_{\text{Linear term}}$$

Quadratic coefficients

Subject to

$$\sum_{i=1}^N \underbrace{\alpha_i y^{\{i\}}}_{\text{Linear equality constraint}} = y^T \alpha = 0$$

Linear equality constraint

Pass these to a quadratic programming package

$$\text{lower bound}(0) \leq \alpha \leq \text{upper bound}(\infty)$$

$$\min_{\alpha} \frac{1}{2} \alpha^T Q \alpha - 1^T \alpha \quad \text{subject to} \quad y^T \alpha = 0; \alpha \geq 0$$

$\uparrow$   
 Quadratic  
 coefficient

Quadratic programming will give us  $\alpha$

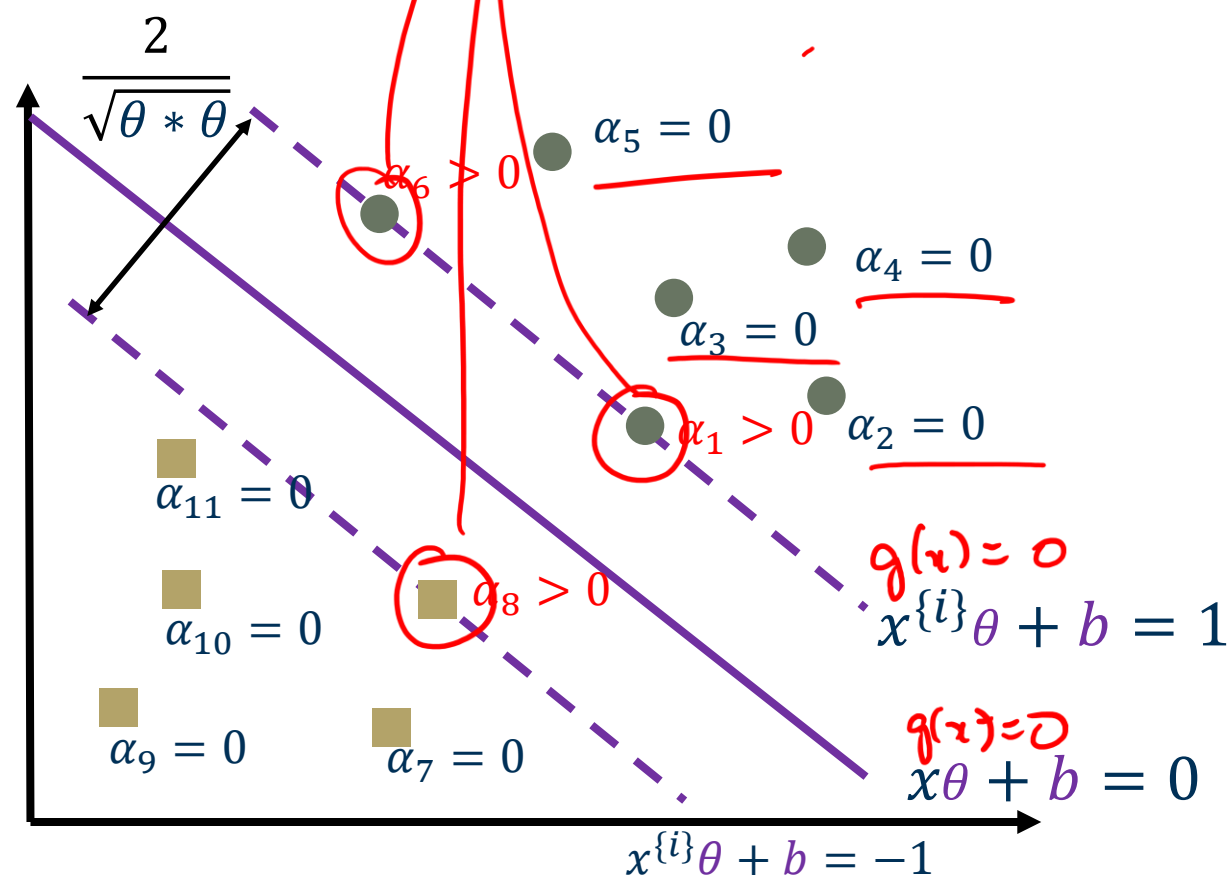
Solution:  $\alpha = \alpha_1, \dots, \alpha_N$

KKT condition ( $\alpha_i g_i(\theta) = 0$ ):

$$\alpha_i (y^{\{i\}} (x^{\{i\}} \theta + b) - 1) = 0$$

$$(y^{\{i\}} (x^{\{i\}} \theta + b) - 1) > 0 \Rightarrow \alpha_i = 0$$

$$(y^{\{i\}} (x^{\{i\}} \theta + b) - 1) = 0 \Rightarrow \alpha_i > 0 \Rightarrow x_i \text{ is a support vector}$$



- Class 2
- Class 1

# Training

$$\theta = \sum_{i=1}^N \alpha_i y^{(i)} x^{(i)}$$

Quadratic Programming

No need to go over all datapoints

$$\rightarrow \theta = \sum_{\underline{x^{(i)} \text{ in SV}}} \alpha_i y^{(i)} x^{(i)}$$

and for  $b$  pick any support vector and calculate:

$$y^{(i)}(x^{(i)}\theta + b) = 1$$

Linearly Separable Data

# Testing

For a new test point  $s$

Compute:

$$s\theta + b = \sum_{\underline{x^{(i)} \text{ in SV}}} \alpha_i y^{(i)} x^{(i)} s^T + b$$

Classify  $s$  as class 1 if the result is positive, and class 2 otherwise

# Why primal $\rightarrow$ dual form?

## Primal Form

$$\text{Minimize} \quad \frac{1}{2} \theta \theta^T \quad \text{s.t.} \quad y^{(i)} (x^{(i)} \theta + b) - 1 \geq 0$$

## Dual Form

$$\min_{\alpha} \frac{1}{2} \alpha^T \begin{bmatrix} y^{(1)} y^{(1)} x^{(1)} x^{(1)T} & \dots & y^{(1)} y^{(N)} \boxed{x^{(1)} x^{(N)T}} \\ \vdots & \ddots & \vdots \\ y^{(N)} y^{(1)} x^{(N)} x^{(1)T} & \dots & y^{(N)} y^{(N)} x^{(N)} x^{(N)T} \end{bmatrix} \alpha + (-I^T) \alpha$$

$n \times d$        $d \times n$   
 $K(x) \cdot K(x^T)$

$$\text{s.t.} \quad \sum_{i=1}^N \alpha_i y^{(i)} = y^T \alpha = 0 \quad \text{s.t.} \quad \alpha \geq 0$$

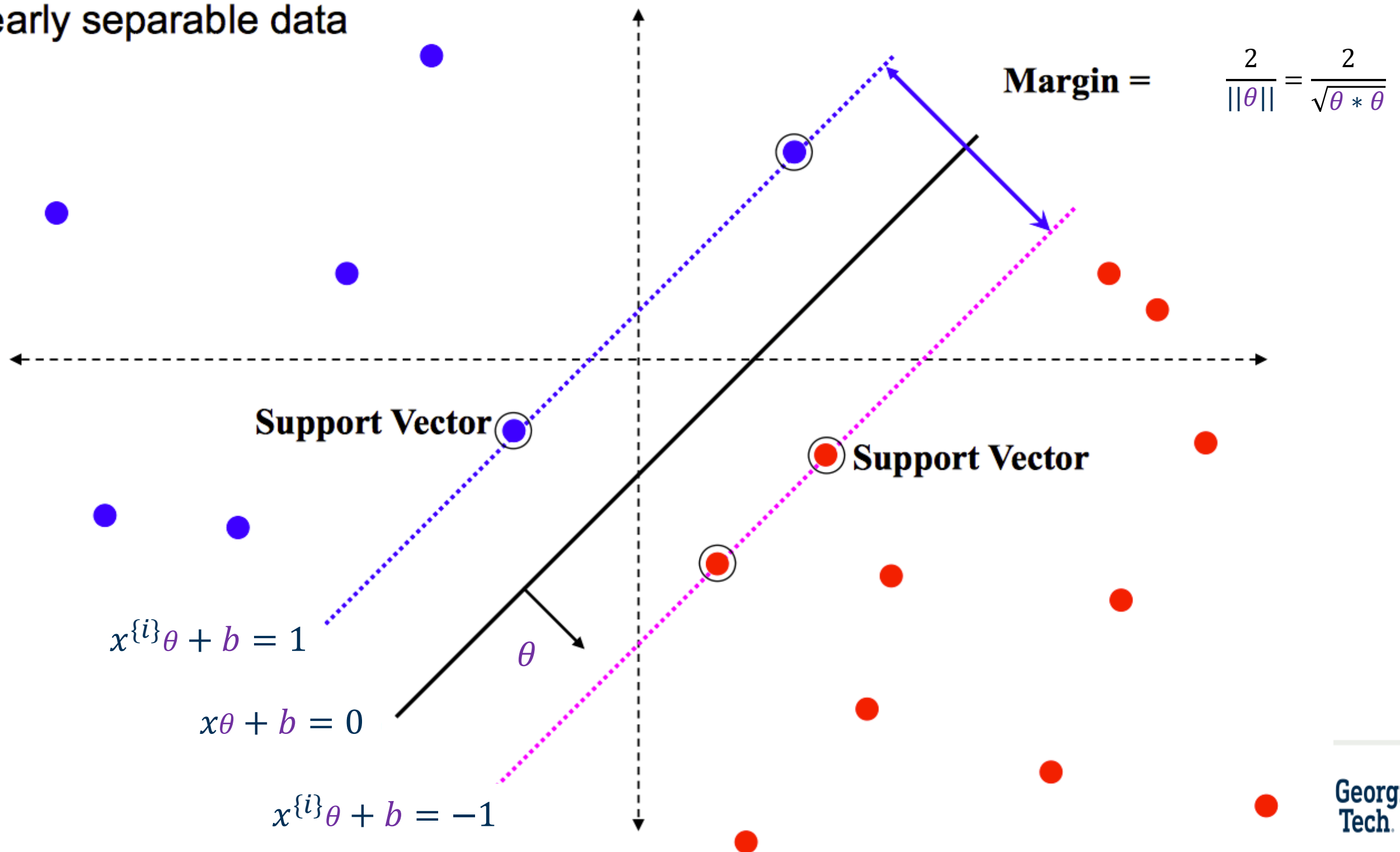
# Why primal $\rightarrow$ dual form?

- **Constraints become easier**
  - Primal has **N inequality constraints**. Dual replaces constraints with  $\alpha_i \geq 0$  and one equality constraint.
  - Optimization becomes a standard **Quadratic Program (QP)**.
- **Only support vectors matter**
  - Dual solution expresses  $\theta = \sum_{i=1}^N \alpha_i y^{(i)} x^{(i)}$
  - Most  $\alpha_i = 0 \rightarrow$  **training depends only on support vectors**, not all data.
- **Enables the kernel trick**
  - Dual objective depends only on **dot products**:  $x^{(i)} \cdot x^{(j)}$
  - Replace dot product with kernel.  $K(x^{(i)}, x^{(j)}) = \phi(x^{(i)})^T \phi(x^{(j)})$
  - Lets SVM work in **very high-dimensional (even infinite-dimensional)** spaces to solve the issue that comes with hard SVM
    - works for only linearly separable data
- **Strong duality holds**
- Dual optimum = primal optimum  $\rightarrow$  solving the dual is **equivalent** and easier.



# Geometric Interpretation

linearly separable data

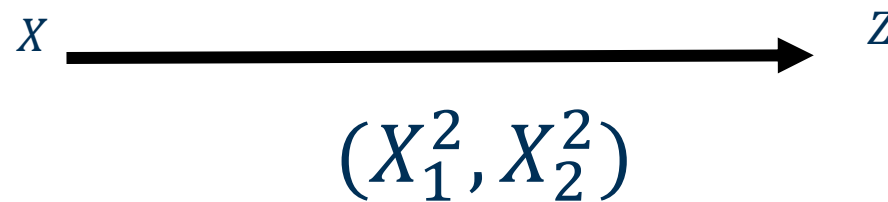
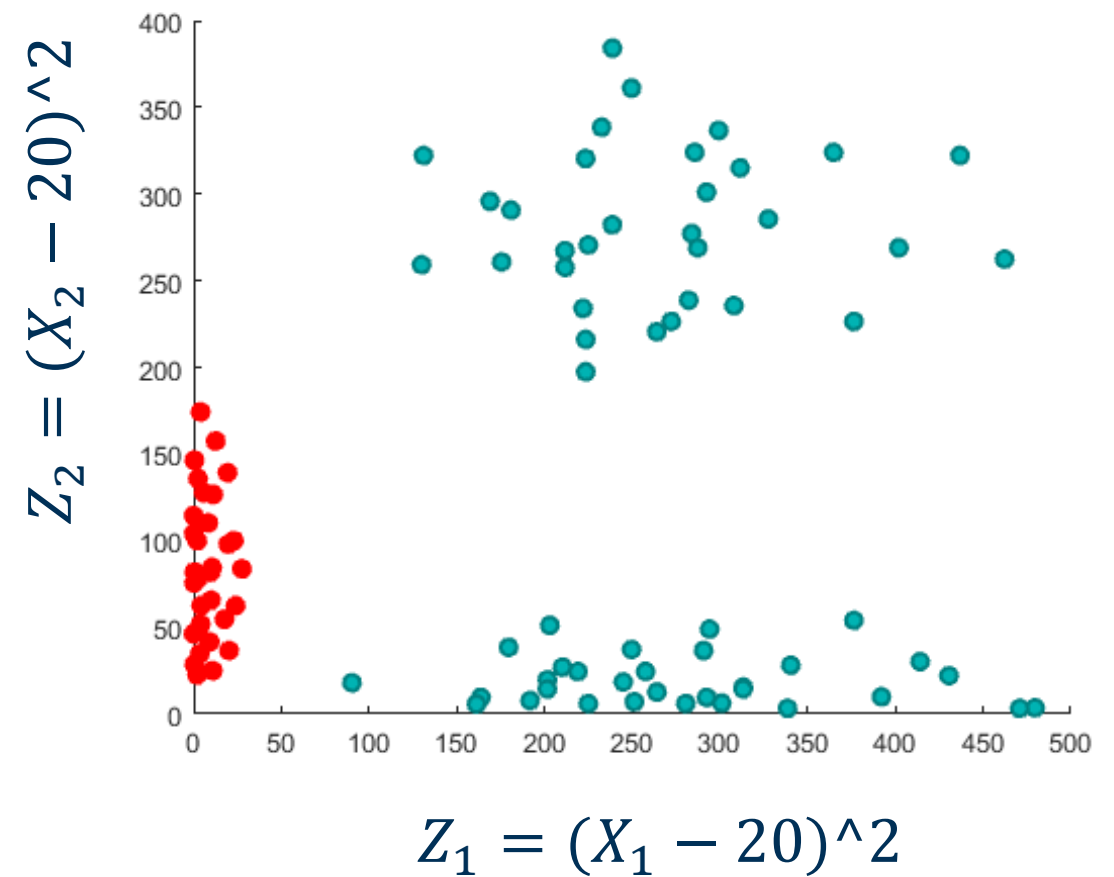
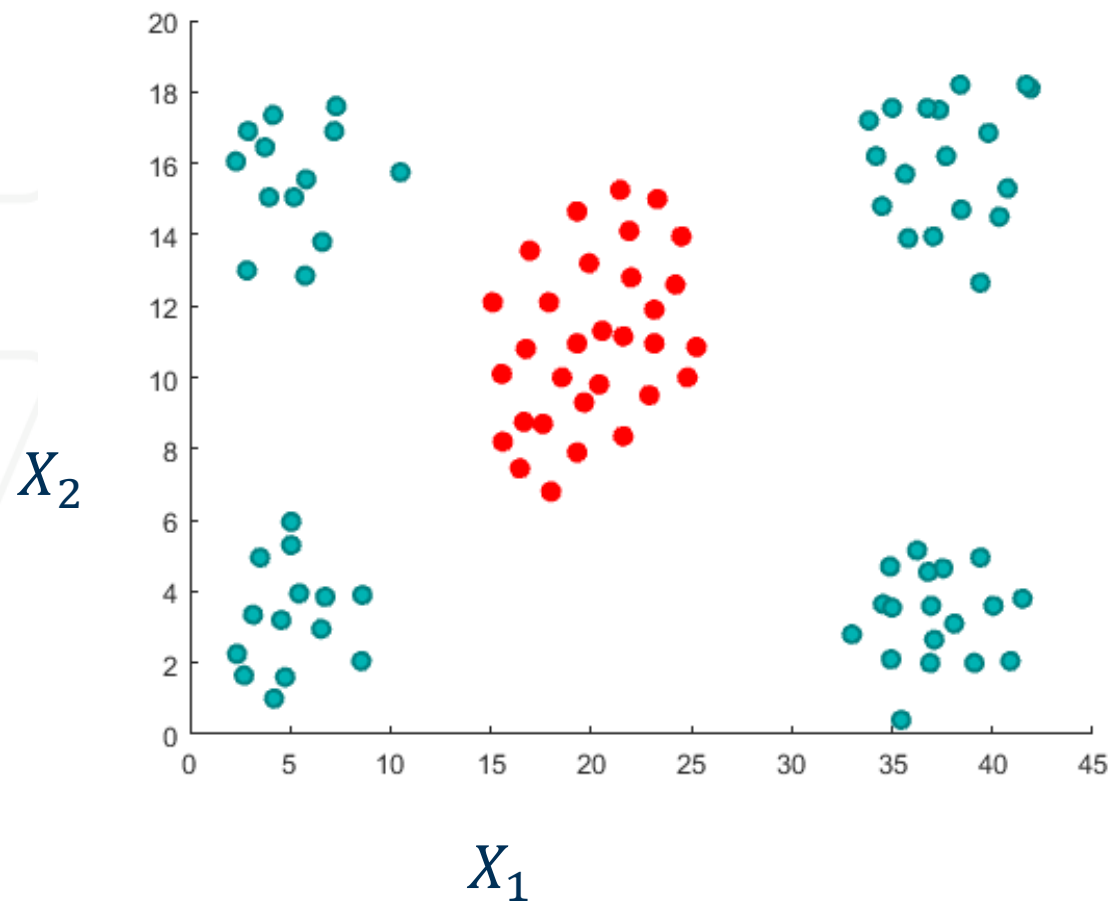




# Take-Home Messages

- Linear Separability
- Perceptron
- SVM: Geometric Intuition and Formulation

# From $x$ to $z$ space



In  $x$  space

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y^{(i)} y^{(j)} \alpha_i \alpha_j \underline{x^{(i)} x^{(j)T}}$$

let's say  $x$  is  $n \times d$   
 $xx^T$  will be  $n \times n$

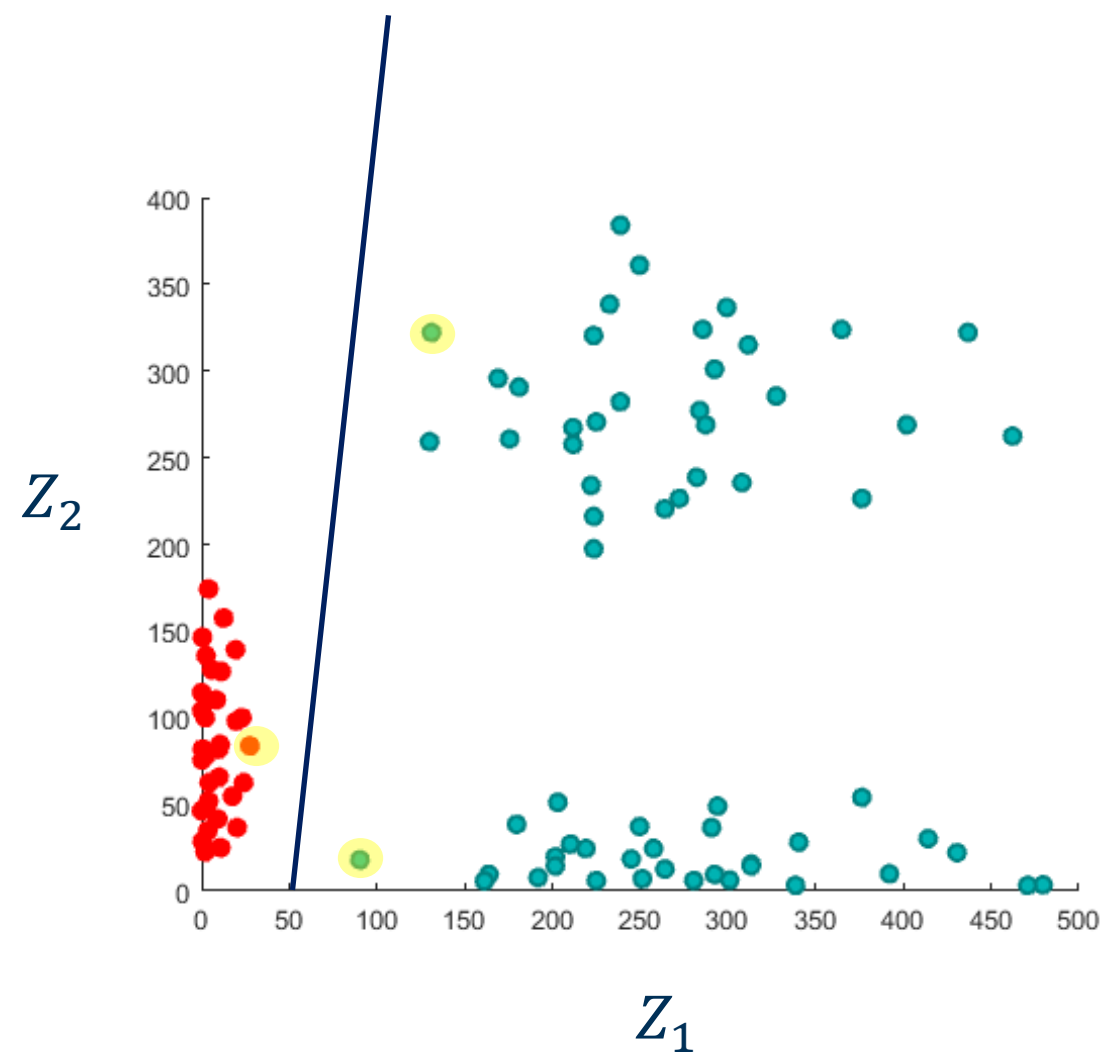
$$_{X_2} \max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y^{(i)} y^{(j)} \alpha_i \alpha_j x^{(i)} x^{(j)T}$$

$X_1$

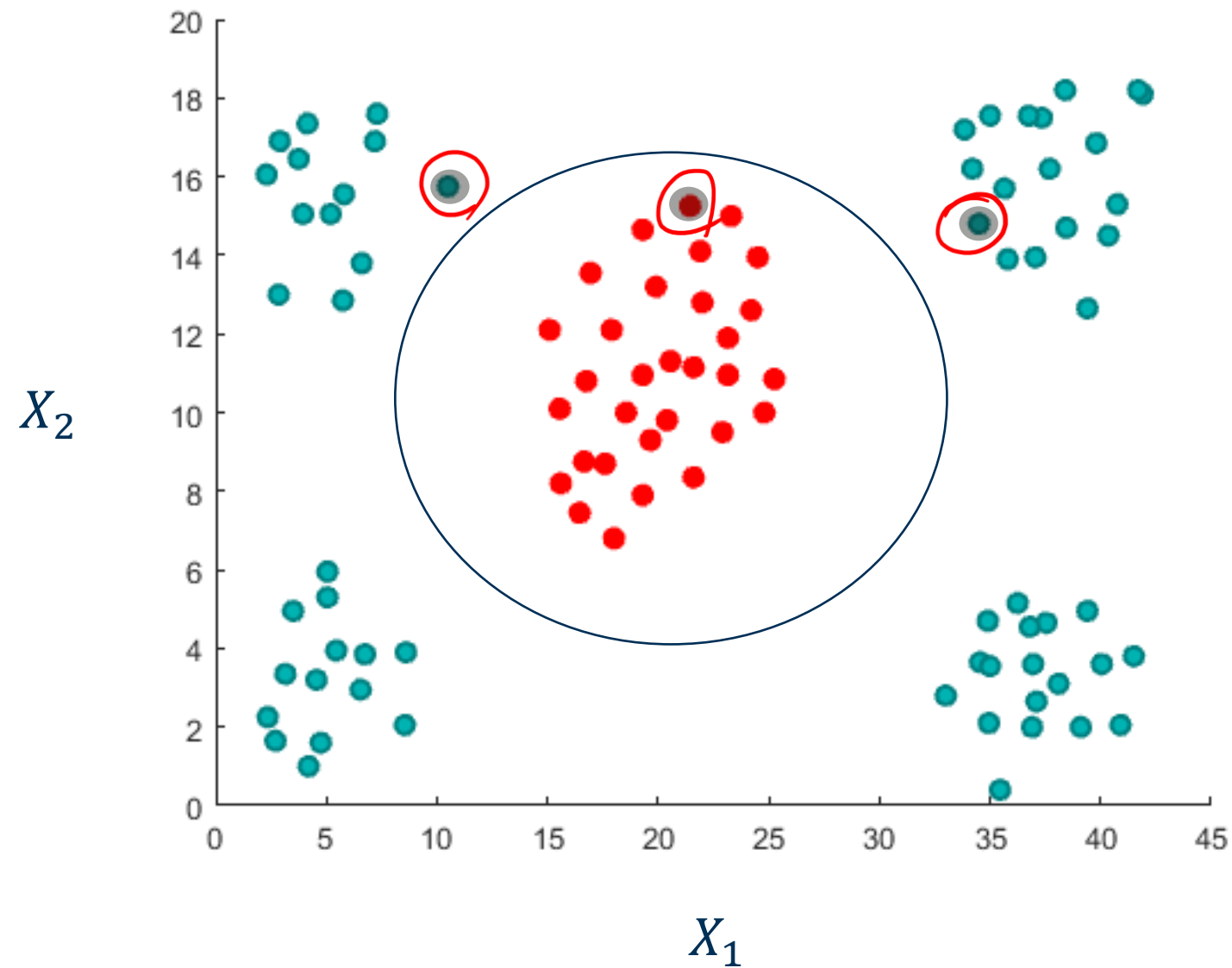
If I add millions of  
dimensions to  $x$ ,  
would it affect the  
final size of  $xx^T$ ?

In  $z$  space

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y^{(i)} y^{(j)} \alpha_i \alpha_j z^{(i)} z^{(j)T}$$



In  $x$  space, they are called pre-images of support vectors



$x$

$y$

# Quick Knowledge Check

- What is the main motivation for SVM compared to perceptron? A. Removes bias from the classifier B. Finds the separating hyperplane with the largest margin C. Forces all margins to be equal D. Ensures nonlinear boundaries
- To enforce correct classification with margin, SVM requires: A.  $y^{(i)}(x^{(i)}\theta + b) \leq 0$  B.  $x^{(i)}\theta + b \geq 0$  C.  $y^{(i)}\theta = 1$  D.  $y^{(i)}(x^{(i)}\theta + b) \geq 1$  sv
- The primal SVM objective is: A. Maximize  $\theta^T\theta$  B. Maximize the number of support vectors C. Minimize  $(1/2)\theta^T\theta$  D. Minimize  $x^{(i)}\theta$  nsv
- Why do we convert the primal SVM to the dual? A. To avoid support vectors B. To increase dimensionality C. To allow kernels and handle constraints more easily D. To remove  $\theta$  entirely
- In testing with SVM, the classification of a point  $s$  is based on: A.  $\sum \alpha_i$  only B.  $b$  only C. distance to nearest sample D.  $\text{sign}(\theta^T s + b)$
- When is a training point a support vector? A. When  $y^{(i)}(x^{(i)}\theta + b) > 2$  B. When  $\alpha_i = 0$  C. When  $y^{(i)}(x^{(i)}\theta + b) = 1$  D. When  $x^{(i)}$  lies inside the margin



# Remaining things in the semester

- Kernel SVM will be released. **Please go through it.**
  - How to use kernel trick to use SVM for non linearly separable data
  - Take data from  $x$  space to  $z$  space (higher dimension)
  - High chances of data to be linearly separable in  $z$  space
  - Do SVM in that space
  - Use kernel trick
- Quiz 13 & 14 – This and next weekend
- Final Presentation with your mentor – Link on Ed
- Final project – Due Dec 2<sup>nd</sup>
- Peer Evaluation – Due Dec 2<sup>nd</sup>



**Please fill out CLOS. Potential Extra Credit Opportunity  
will be released on Ed**

**Thank you for a wonderful semester!😊**

**Good Luck in completing your projects and for all the success you are about to achieve!**





**colleague  
congratulating me  
for achieving  
state-of-the-art**

**me who trained  
on test data**